

ARRAYS 2

"Goals Begin Behaviors –
Consequences Maintain Behavior"



Agenda

01. Find in row wise & colwise sorted matrix
02. Row with maximum no. of 1s
03. Sum of all submatrices sum.

Q1. Given a row wise & col wise sorted matrix.
Search for an ele k

A =

-5	-2	1	13
-4	0	3	14
-3	2	6	18

$k=13 \rightarrow \text{True}$

$k=2 \rightarrow \text{True}$

$k=7 \rightarrow \text{False}$

BF Approach $\rightarrow$ Iterate on all the cells, iterate on entire matrix.

TC: $O(n*m)$

SC: $O(1)$

Idea 2 $\rightarrow$ Start from right most cell of first row

A =

-5	-2	1	13
-4	0	3	14
-3	2	6	18

$k=0$

return true

	0	1	2	m-1
0	-3	-2	-1	0
1	2	3	7	8
2	7	10	12	13
3	8	14	17	20
4	21	30	32	33

$n \times m$

$$K = 15$$

Logic $\rightarrow A[i] > K \rightarrow$ move left
(discard col)

$A[i] < K \rightarrow$ move down
(discard row)

$$i = 0, j = m-1$$

while ($i < n$ & $j \geq 0$) {

if ($A[i][j] > K$) $j = j - 1$

else if ($A[i][j] < K$) $i = i + 1$

else return true;

}

return false;

Tc: $O(n+m)$

Sc: $O(1)$

Q2 Given a binary sorted matrix. Find the row with maximum no. of 1's.

$A = \begin{bmatrix} [0, 1, 1] \\ [0, 0, 1] \\ [0, 1, 1] \end{bmatrix} \longrightarrow \text{Ans} = 0^{\text{th}} \text{ row}$

$A = \begin{bmatrix} [0, 0, 0, 0] \\ [0, 0, 0, 1] \\ [0, 0, 1, 1] \\ [0, 1, 1, 1] \end{bmatrix} \longrightarrow \text{Ans} = 3^{\text{rd}} \text{ row}$

* Brute force $\rightarrow$ Iterate on each & every row calculating the no. of 1s

Tc: $O(n*m)$

Sc: $O(1)$

	0	1	2	$m-1$
0	0	0	1	1
1	0	0	0	1
2	0	0	0	0
3	0	1	1	1
4	0	0	0	1

$n \times m$

$i = 0 \quad j = m - 1$

$ans = \emptyset \quad 3$

while ($i < N \ \&\& \ j \geq 0$) {

 while ($j \geq 0 \ \&\& \ A[i][j] == 1$) {

$j = j - 1;$

$ans = i;$

$i++;$

T.C: $O(n+m)$

S.C: $O(1)$

3

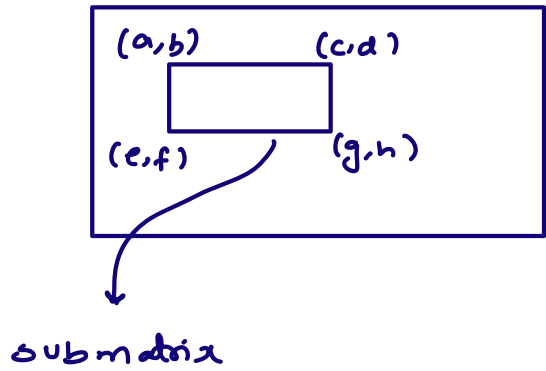
	0	1	2	3
0	0	0	1	1
1	0	0	0	1
2	0	1	1	1
3	0	1	1	1
4	0	0	0	1

n

Ans = 3

* $\rightarrow$ move from bottom
right

Submatrix : Part of a matrix



01. Single cell is also a submatrix

02. Complete matrix is also a submatrix

To represent a submatrix

↳ TL & BR

↳ TR & BL

03. Given a matrix of N rows & M columns. determine the sum of all possible submatrices **sum**

$$\begin{array}{c} 0 \\ 1 \\ 2 \end{array} \begin{bmatrix} 3 & 1 \\ -1 & -2 \\ 2 & 4 \end{bmatrix} = \left\{ \begin{array}{l} \begin{bmatrix} 3 \end{bmatrix} \quad \begin{bmatrix} 3, 1 \end{bmatrix} \quad \begin{bmatrix} 3 & 1 \\ -1 & -2 \end{bmatrix} \quad \begin{bmatrix} 3 & 1 \\ -1 & -2 \\ 2 & 4 \end{bmatrix} \quad \begin{bmatrix} 3 \\ -1 \end{bmatrix} \quad \begin{bmatrix} 3 \\ -1 \\ 2 \end{bmatrix} \\ \begin{bmatrix} 1 \end{bmatrix} \quad \begin{bmatrix} 1 \\ -2 \end{bmatrix} \quad \begin{bmatrix} 1 & -2 \\ -2 & 4 \end{bmatrix} \quad \begin{bmatrix} -1 \end{bmatrix} \quad \begin{bmatrix} -1 & -2 \end{bmatrix} \quad \begin{bmatrix} -1 \\ 2 \end{bmatrix} \quad \begin{bmatrix} -1 & -2 \\ 2 & 4 \end{bmatrix} \\ \begin{bmatrix} 2 \end{bmatrix} \quad \begin{bmatrix} 2 & 4 \end{bmatrix} \quad \begin{bmatrix} 4 \end{bmatrix} \quad \begin{bmatrix} -2 \end{bmatrix} \quad \begin{bmatrix} -2 \\ 4 \end{bmatrix} \end{array} \right.$$

Ans = 36

* BF Approach →

01 Generate all submatrices

↳ for one submatrix, iterate & get sum

↳ totalsum += sum;

TL & BR

for (i=0; i<n; i++) {

for (j=0; j<m; j++) {

TL = i, j

for (k=i; k<n; k++) {

for (l=j; l<m; l++) {

TL = i, j & BR = k, l

Sum of submatrix

TL

BR

Tc: $O(n^3 * m^3)$

Sc: $O(1)$

Contribution Technique

arr[i] * contribution

$$3 * 6 = 18$$

$$1 * 6 = 6$$

$$-1 * 8 = -8$$

$$-2 * 8 = -16$$

$$2 * 6 = 12$$

$$4 * 6 = 24$$

36

* How many no. of times is (2,3) going to contribute?

	0	1	2	3	4
0	✓	✓	✓	✓	
1	✓	✓	✓	✓	
2	✓	✓	✓	✓	✓
3				✓	✓
4				✓	✓

Possible options for

TL * BR

$$12 * 6 = \underline{72}$$

	0	1	2	3	4
0	✓	✓			
1	✓	✓ ✓	✓	✓	✓
2		✓	✓	✓	✓

3 * 5

In how many submatrices will this $(1,1)$ be present
 i, j

$$TL = (i+1) * (j+1)$$

$$BR = (n-i) * (m-j)$$

$$count_i = \underbrace{(i+1) * (j+1)}_{TL} * \underbrace{(n-i) * (m-j)}_{BR}$$

```
for (i=0; i<n; i++)
```

```
    for (j=0; j<m; j++) {
```

```
        count_i = (i+1) * (j+1) * (n-i) * (m-j)
```

```
        ans = ans + arr[i][j] * count_i;
```

```
    }
```

```
}
```

$$TC: O(n * m)$$

$$SC: O(1)$$